

Reinforcement Learning II

Giulio Turrisi, Postdoctoral Researcher
Italian Institute of Technology

- Tabular methods are “exact”: you can obtain the **optimal policy**
- ...but they cannot be used for large scale problem
- DQN solved this for discrete actions problems (e.g. videogames)
- PPO did an additional step onwards, and made continuous actions problems tractable

Off-Policy

Pro

Sample-efficient – older sample are not discarded

Cons

High GPU VRAM – maintaining samples is memory consuming

On-policy

Pro

Low GPU VRAM – older sample are discarded

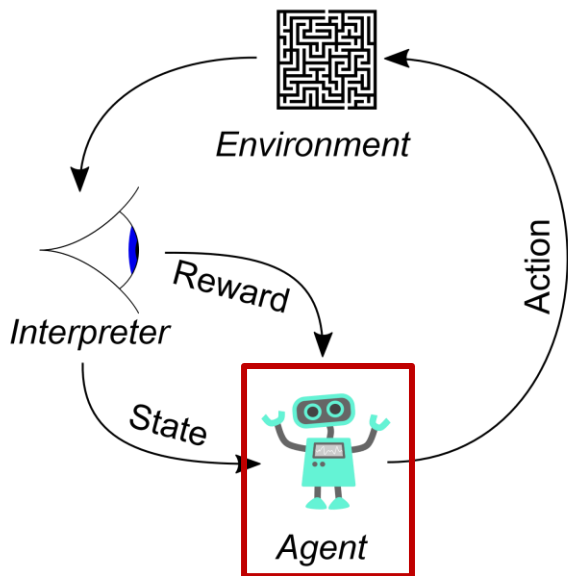
Cons

Not Sample-efficient – older sample are discarded – need to generate new data every time

- Environments
- Observations
- Reward Functions
- From Simulators to Real World

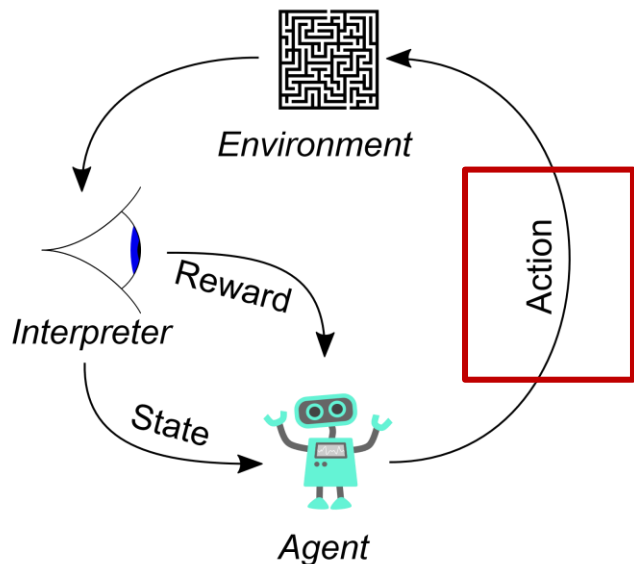
- **Environments**
 - Simulators
 - Computing
- Observations
- Reward Functions
- From Simulators to Real World

RL involves agents **learning** by **trial and error**, interacting with an **environment**, and maximizing cumulative **reward**



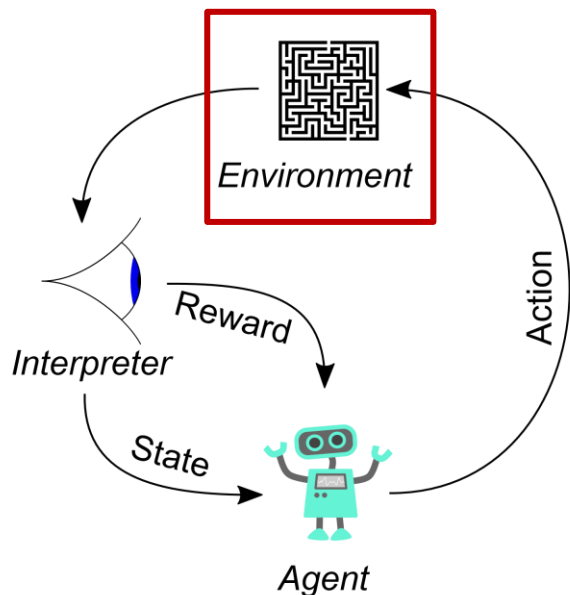
- **Agent:** Robots

RL involves agents **learning** by **trial and error**, interacting with an **environment**, and maximizing cumulative **reward**



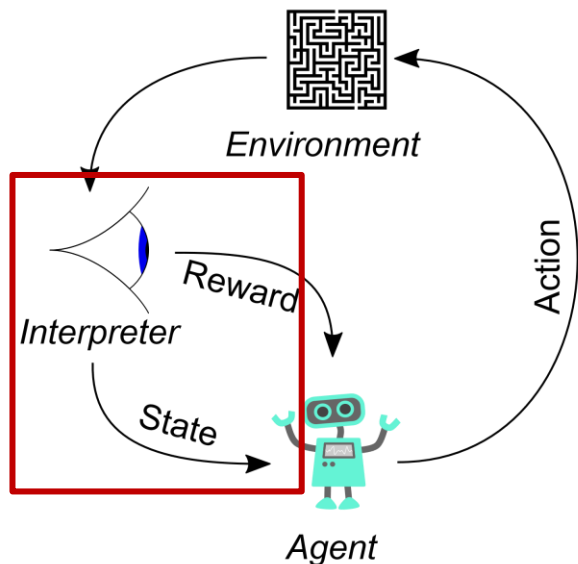
- **Agent:** Robots
- **Action:** Motor desired position

RL involves agents **learning** by **trial and error**, interacting with an **environment**, and maximizing cumulative **reward**



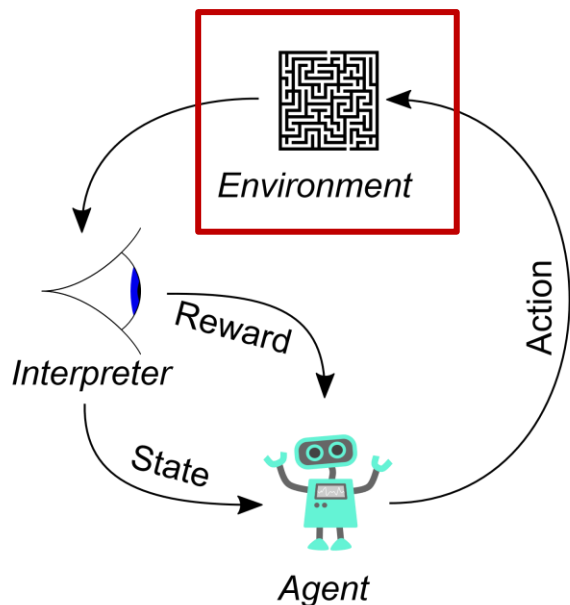
- **Agent**: Robots
- **Action**: Motor desired position
- **Environment**: Real world or simulators

RL involves agents **learning** by **trial and error**, interacting with an **environment**, and maximizing cumulative **reward**



- **Agent**: Robots
 - **Action**: Motor desired position
- **Environment**: Real world or simulators
 - **New State**: sensors information
- **Rewards**: Signal on how good the action was

RL involves agents **learning** by **trial and error**, interacting with an **environment**, and maximizing cumulative **reward**



- **Agent:** Robots
- **Action:** Motor desired position
- **Environment:** Real world or simulators
- **New State:** sensors information
- **Rewards:** Signal on how good the action was

Where can we train our policy?

In order to learn by trial and error, a robot needs to explore!



- **Agent:** Cart Pendulum
- **Action:** Motors torque
- **Environment:** Real world
- **Rewards:** +1 if upright position

It is **unfeasible** to learn complicate task directly in the real world! (**years of experience needed – robot destroyed!**)

What if we train a quadruped locomotion in the real world?



- **Agent:** Quadruped
- **Action:** Motors Joint Position
- **Rewards:** velocity tracking...more on this later

2-3 years of real-world experience
Too much time, too dangerous.

What if we train a quadruped to use a ladder in the real world?

- **Agent:** Quadruped
 - **Action:** Motors Joint Position
- Rewards:** position goal...more on this later

More **dangerous**.

What if we train a humanoid to use a ladder in the real world?

Sparse 3D structures present a difficult setting

- **Agent:** Humanoid
 - **Action:** Motors Joint Position
- Rewards:** position goal...more on this later

Definitely **too dangerous**.

- In **model-free RL**, you mainly train your policy in simulators
- These simulators need to be:
 - fast (you do not want to wait 3 years for a result!)
 - physically-accurate, to then deploy the policy in the real world without changes

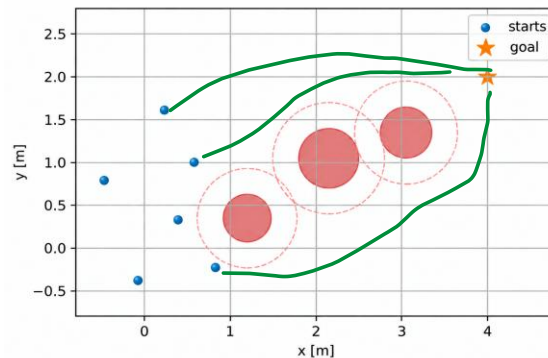
- These simulators need to be:
 - fast (you do not want to wait 3 years for a results!)

Vanilla Policy Gradient

```

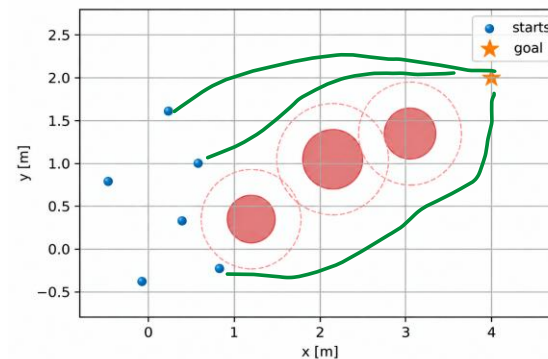
1: Initialize policy parameters  $\theta$ 
2: for iteration = 1, 2, ... do
3:   Collect  $N$  trajectories by executing  $\pi_\theta$ 
4:   for each trajectory  $i = 1, \dots, N$  do
5:     for each timestep  $t = 0, \dots, T_i - 1$  do
6:       Compute the return:
7:        $R_t^{(i)} = \sum_{k=t}^{T_i-1} \gamma^{k-t} r_k^{(i)}$ 
8:     end for
9:   end for
10:  Compute the policy-gradient estimate:
11:   $\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_\theta \log \pi_\theta \left( a_t^{(i)} \mid s_t^{(i)} \right) R_t^{(i)}$ 
12:  Update the policy:
13:   $\theta \leftarrow \theta + \alpha \hat{g}$ 
14: end for
    
```

Initialize the robot in different initial state, and execute the policy, e.g.



- These simulators need to be:
 - fast (you do not want to wait 3 years for a result!)
- Two main conditions:
 1. **Parallelizable** (each trajectory can be a separate robot running in parallel)
 2. Each trajectory should be collected **as fast as possible**

Initialize the robot in different initial state, and execute the policy, e.g.





With audio

Learning quadrupedal locomotion over challenging terrain

Joonho Lee¹, Jemin Hwangbo^{1,2†}, Lorenz Wellhausen¹,
Vladlen Koltun³, Marco Hutter¹

¹ Robotic Systems Lab, ETH Zurich

² Robotics & Artificial Intelligence Lab, KAIST

³ Intelligent Systems Lab, Intel

†Substantial part of the work was carried out during his stay at 1

ETH zürich

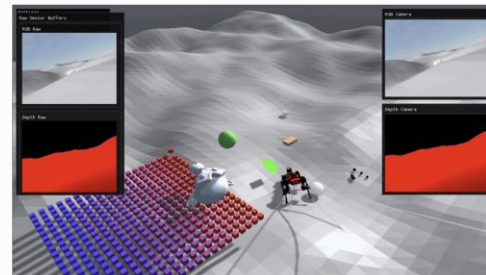


Trained in 12 hours, in RAISIM



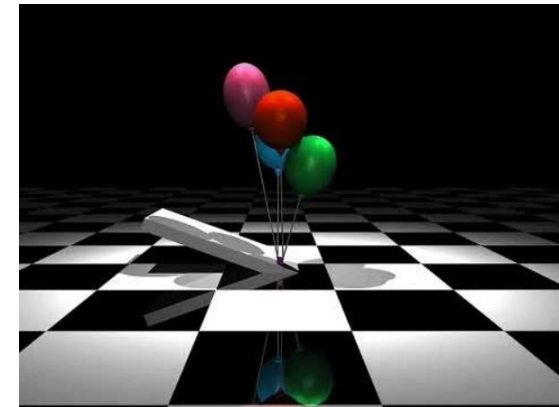
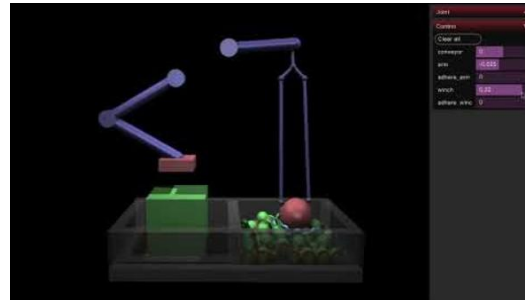
RaiSim v2.5.7

RaiSim v2.5.7



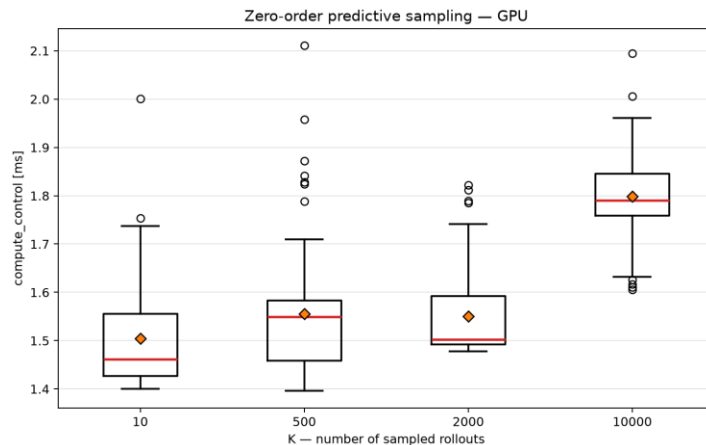
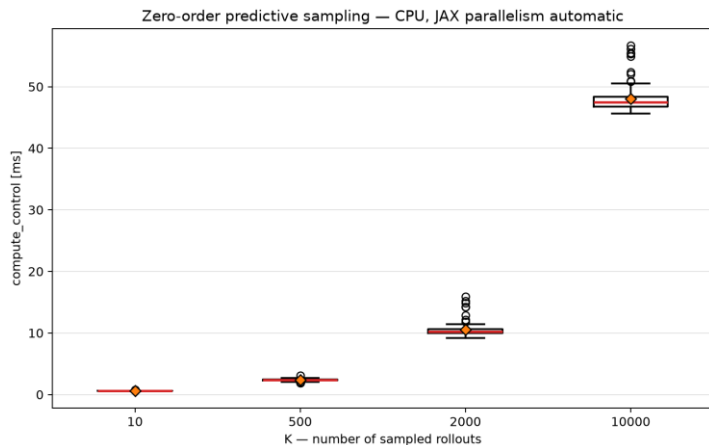
One of the fastest CPU simulators out there. Limits: **number of CPU cores in your machine**

- Created by Emo Todorov, Tom Erez, Yuval Tassa (2012!)
- Recently acquired by Google Deepmind, Open Source
- One of the fastest CPU simulators out there
- Soft contact modelling

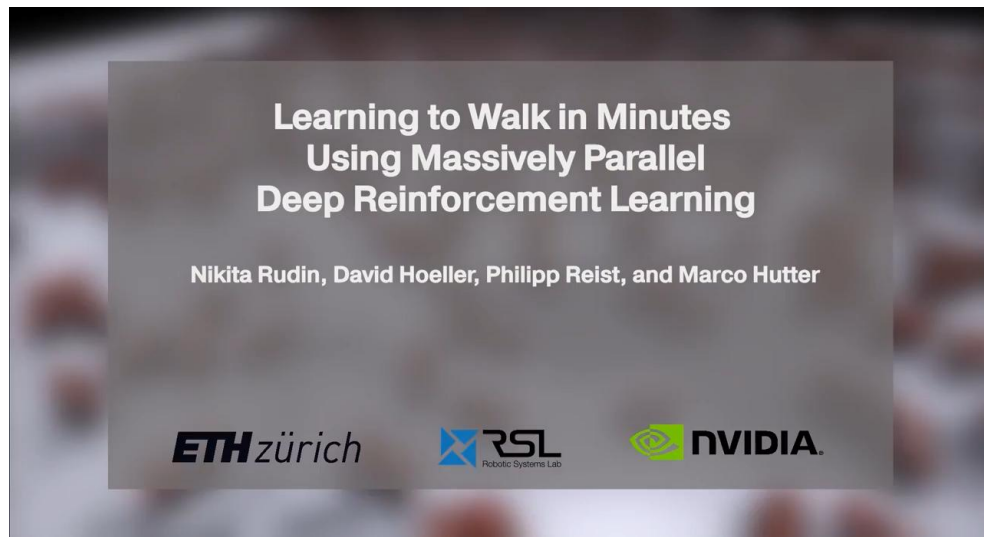


CPU vs GPU Simulators

Remember what we show in the sampling-based control class:



If the number of samples in parallel is low, CPU is the best. Otherwise use GPU



Trained in 20-30 min, in IsaacGym

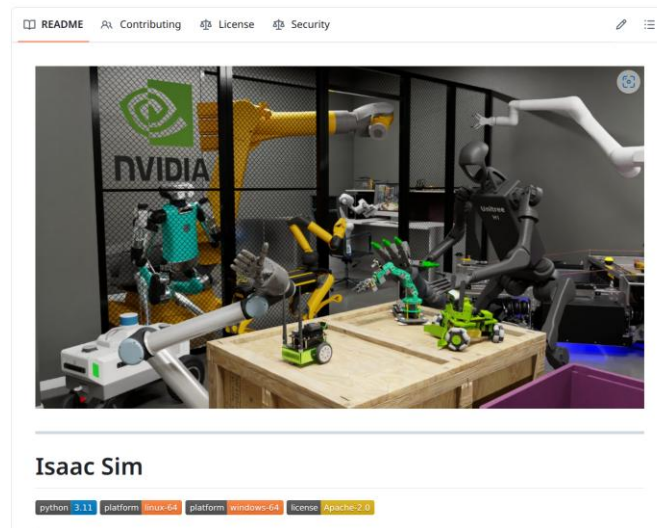


≥ 4096 robots in parallel
Physics backend: **PhysX**

- IsaacGym was discontinued in 2022
- New simulator: **IsaacSim/IsaacLab**
- Physics backend:
 - **PhysX** (like IsaacGym)
 - **"Newton"**



Started by Nvidia, Disney, and Deepmind (Mujoco)
Contains multiple physics engine: **Mujoco**, **Kamino**, etc..



<https://github.com/isaac-sim/IsaacSim>

<https://github.com/isaac-sim/IsaacLab>

If one want to use **mujoco warp directly**

- Simulator: **MjLab**
- Sami API of IsaacLab
- Support mujoco warp only



<https://github.com/mujocolab/mjlab>

- Runs on AMD, Nvidia, Apple Silicon

README Apache-2.0 license



Genesis World

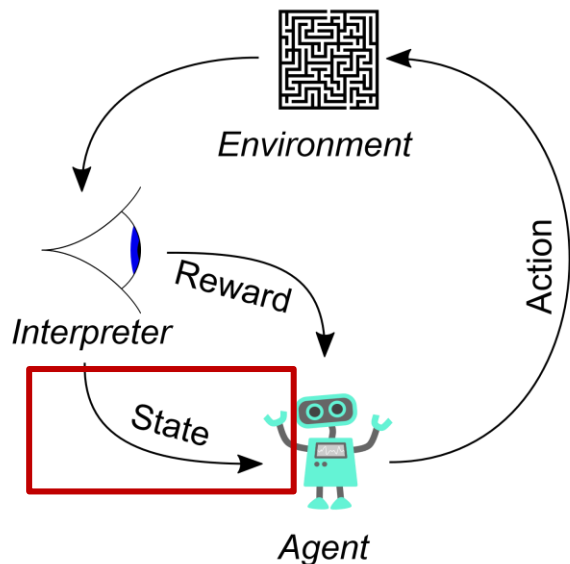
pypi v1.3.3 downloads 622k docs passing issues 106 open discussions 316 total

Genesis World is a simulation platform for physical AI developments. It combines a unified multi-physics engine, a photo-realistic renderer ([Nyx](#)), and a cross-platform compiler ([Quadrants](#)) behind a Pythonic simulation interface. Genesis World is designed to scale from a single laptop kernel to datacenter-grade GPUs, while remaining easy to read, extend, and embed in research code.

<https://github.com/Genesis-Embodied-AI/genesis-world>

- Environments
- **Observations**
 - Most used observations in locomotion
- Reward Functions
- From Simulators to Real World

RL involves agents **learning** by **trial and error**, interacting with an **environment**, and maximizing cumulative **reward**



- **Agent:** Robots
- **Action:** Motor desired position
- **Environment:** Real world or simulators
- **New State (obs):** sensors information
- **Rewards:** Signal on how good the action was

The feedback of your policy

Typical observation for quadruped legged locomotion

- Base linear velocity $\in R^3$
- Base Angular velocity $\in R^3$
- Base Orientation $\in R^3$
- Desired Base Orientation $\in R^2$
- Last Action $\in R^{12}$
- Joint positions $\in R^{12}$
- Joint velocity $\in R^{12}$

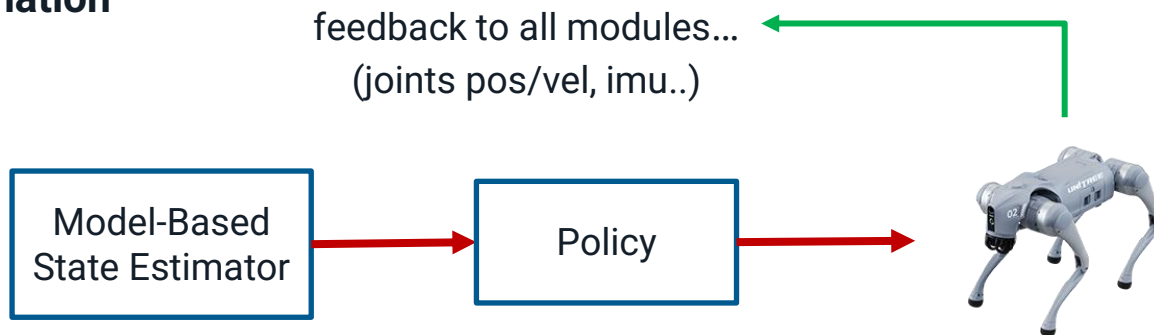
Typical observation for quadruped legged locomotion

- Base linear velocity $\in R^3$
- Base Angular velocity $\in R^3$
- Base Orientation $\in R^3$
- Desired Base Orientation $\in R^2$
- Last Action $\in R^{12}$
- Joint positions $\in R^{12}$
- Joint velocity $\in R^{12}$

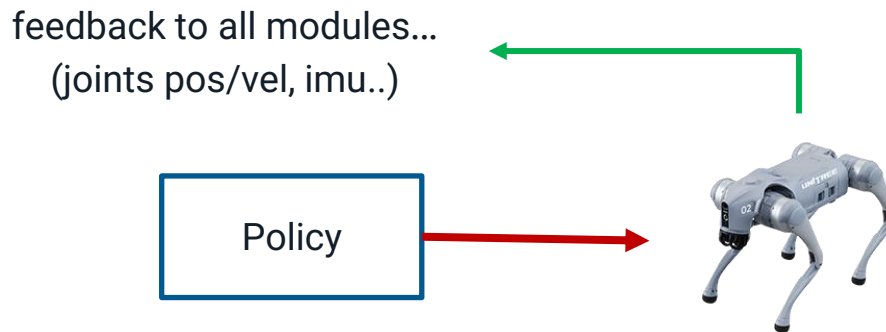


1. from State Estimation
2. use imu acceleration
3. Learned State Estimation

1. from State Estimation



2. use IMU linear acceleration



Typical observation for quadruped legged locomotion

- Base linear velocity $\in R^3$
- Base Angular velocity $\in R^3$
- Base Orientation $\in R^3$
- Desired Base Orientation $\in R^2$
- Last Action $\in R^{12}$
- Joint positions $\in R^{12}$
- Joint velocity $\in R^{12}$



1. from State Estimation

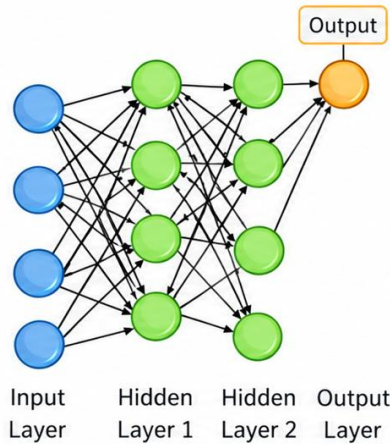
2. use imu acceleration

3. Learned State Estimation

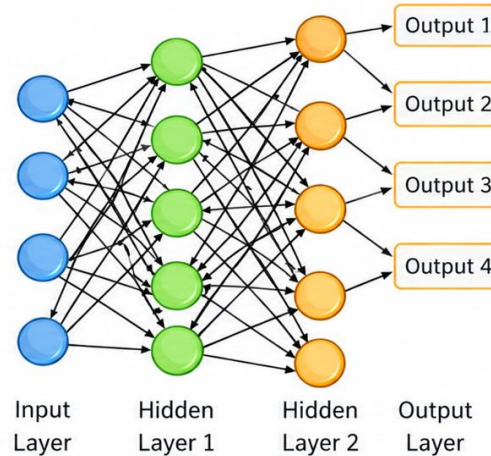
Idea:

Learn, together with the policy, **an additional neural network** to estimate linear velocity **from IMU + proprioceptive**

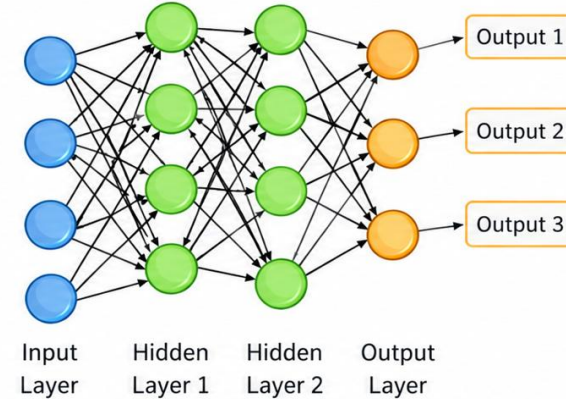
Learned State Estimation



Critic:
Only used in simulation!



Actor:
**Used in simulation
and in deploy**



Learned SE:
**Used in simulation and in
deploy.** Predict Linear velocity
for the Actor

Initialize policy π , critic V , and state estimator SE

for each PPO training iteration:

collect rollouts in simulation:

observe proprioception: o_t

predict linear velocity: $\hat{v}_t = SE(o_t)$

build policy input: $x_t = [o_t, v_t]$

sample action: $a_t = \pi(x_t)$

step simulation and store transition

store ground truth velocity from simulation v_t

compute rewards, returns, and advantages

update PPO using collected rollouts

update estimator with supervised loss: $\|\hat{v}_t - v_t\|$



Personal experience:

If you have a model-based estimator,
use it! **Usually better performance**

open-source implementations:

Model-based SE:

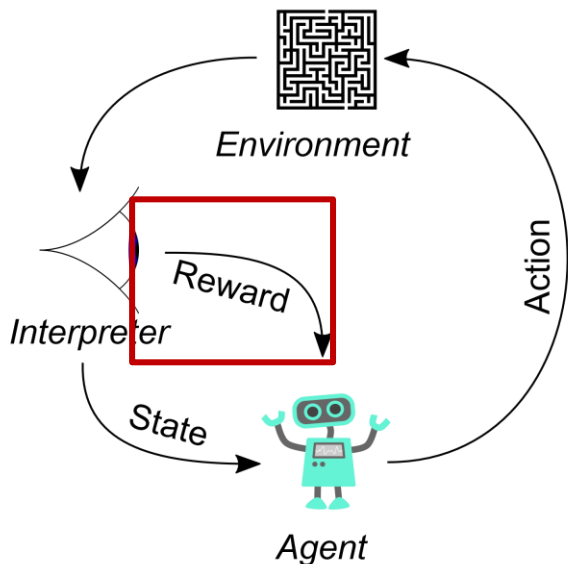
<https://github.com/iit-DLSLab/muse>

Concurrent SE:

<https://github.com/iit-DLSLab/basic-locomotion-isaacsim>

- Environments
- Observations
- **Reward Functions**
 - Most used rewards in locomotion
 - How to walking like a human
- From Simulators to Real World

RL involves agents **learning** by **trial and error**, interacting with an **environment**, and maximizing cumulative **reward**



- **Agent:** Robots
 - **Action:** Motor desired position
- **Environment:** Real world or simulators
 - **New State:** sensors information
- **Rewards:** Signal on how good the action was

How to define a good locomotion behaviour?

Each reward are multiplied by a scalar to increase its importance

Type	Expression
base linear velocity	$\exp(-\ \mathbf{v}^{\text{des}} - \mathbf{v}\ ^2/0.05)$
yaw rate	$\exp(-(w_z^{\text{des}} - w_z)^2/0.25)$
base height	$\exp(-(h_{\text{base}}^{\text{des}} + h^{\text{terrain}} - h_{\text{base}})^2/0.01)$
base pitch orientation	$-(\theta^{\text{terrain}} - \theta)^2$
base roll orientation	$-(\phi^{\text{terrain}} - \phi)^2$
base vertical velocity	$-v_z^2$
joints torque	$-\ \boldsymbol{\tau}\ ^2$
joints acceleration	$-\ \ddot{\mathbf{q}}\ ^2$
joints power	$- \dot{\mathbf{q}} \odot \boldsymbol{\tau} $
action rate	$-\ (\mathbf{a}_k - \mathbf{a}_{k-1})\ ^2$
action smoothness	$-\ (\mathbf{a}_k - 2\mathbf{a}_{k-1} + \mathbf{a}_{k-2})\ ^2$
undesired contact	$-\sum_i 1_{\text{contact},i}$
stance contact suggestion	$\frac{1}{4} \sum_{i=1}^4 c_i$
feet air time	$\sum_{i=1}^4 t_i^{\text{air}}$
feet air-time variance	$-\text{Var}(t_i^{\text{air}}, t_i^{\text{contact}})$
feet slide	$-\sum_{i=1}^4 \ \mathbf{v}_{\text{foot},i}^{xy}\ c_i$
feet height clearance	$\sum_{i=1}^4 \exp(-(\mathbf{p}_{\text{foot}}^{\text{des},z} + h^{\text{terrain}} - \mathbf{p}_{\text{foot}}^z)^2/0.01))$
feet to hips position	$-\sum_{i=1}^4 \ \tilde{\mathbf{p}}_{\text{foot},i}^{xy} - \tilde{\mathbf{p}}_{\text{hip},i}^{xy}\ $
feet vertical surface contact	$-1(\ \mathbf{f}_{xy}\ > 4 f_z)$

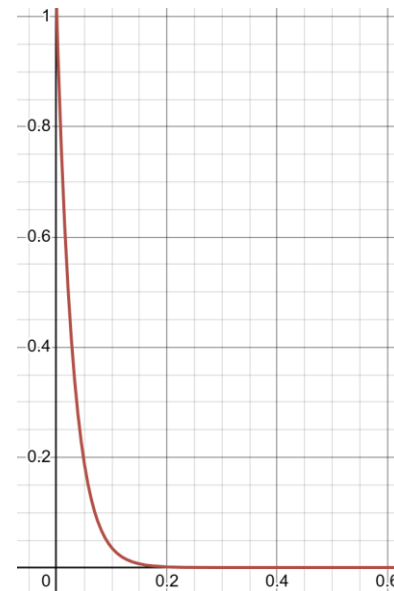


Rewards Engineering
(just for walking)!

Each reward are multiplied by a scalar to increase its importance

Type	Expression
base linear velocity	$\exp(-\ \mathbf{v}^{\text{des}} - \mathbf{v}\ ^2/0.05)$
yaw rate	$\exp(-(w_z^{\text{des}} - w_z)^2/0.25)$
base height	$\exp(-(h_{\text{base}}^{\text{des}} + h^{\text{terrain}} - h_{\text{base}})^2/0.01)$
base pitch orientation	$-(\theta^{\text{terrain}} - \theta)^2$
base roll orientation	$-(\phi^{\text{terrain}} - \phi)^2$
base vertical velocity	$-v_z^2$
joints torque	$-\ \boldsymbol{\tau}\ ^2$
joints acceleration	$-\ \ddot{\mathbf{q}}\ ^2$
joints power	$- \dot{\mathbf{q}} \odot \boldsymbol{\tau} $
action rate	$-\ (\mathbf{a}_k - \mathbf{a}_{k-1})\ ^2$
action smoothness	$-\ (\mathbf{a}_k - 2\mathbf{a}_{k-1} + \mathbf{a}_{k-2})\ ^2$
undesired contact	$-\sum_i 1_{\text{contact},i}$
stance contact suggestion	$\frac{1}{4} \sum_{i=1}^4 c_i$
feet air time	$\sum_{i=1}^4 t_i^{\text{air}}$
feet air-time variance	$-\text{Var}(t_i^{\text{air}}, t_i^{\text{contact}})$
feet slide	$-\sum_{i=1}^4 \ \mathbf{v}_{\text{foot},i}^{xy}\ c_i$
feet height clearance	$\sum_{i=1}^4 \exp(-(\mathbf{p}_{\text{foot}}^{\text{des},z} + h^{\text{terrain}} - \mathbf{p}_{\text{foot}}^z)^2/0.01))$
feet to hips position	$-\sum_{i=1}^4 \ \tilde{\mathbf{p}}_{\text{foot},i}^{xy} - \tilde{\mathbf{p}}_{\text{hip},i}^{xy}\ $
feet vertical surface contact	$-1(\ \mathbf{f}_{xy}\ > 4 f_z)$

Exponential are stronger
than l2 norm



Each reward are multiplied by a scalar to increase its importance

Type	Expression
base linear velocity	$\exp(-\ \mathbf{v}^{\text{des}} - \mathbf{v}\ ^2/0.05)$
yaw rate	$\exp(-(w_z^{\text{des}} - w_z)^2/0.25)$
base height	$\exp(-(h_{\text{base}}^{\text{des}} + h^{\text{terrain}} - h_{\text{base}})^2/0.01)$
base pitch orientation	$-(\theta^{\text{terrain}} - \theta)^2$
base roll orientation	$-(\phi^{\text{terrain}} - \phi)^2$
base vertical velocity	$-v_z^2$
joints torque	$-\ \boldsymbol{\tau}\ ^2$
joints acceleration	$-\ \ddot{\mathbf{q}}\ ^2$
joints power	$- \dot{\mathbf{q}} \odot \boldsymbol{\tau} $
action rate	$-\ (\mathbf{a}_k - \mathbf{a}_{k-1})\ ^2$
action smoothness	$-\ (\mathbf{a}_k - 2\mathbf{a}_{k-1} + \mathbf{a}_{k-2})\ ^2$
undesired contact	$-\sum_i 1_{\text{contact},i}$
stance contact suggestion	$\frac{1}{4} \sum_{i=1}^4 c_i$
feet air time	$\sum_{i=1}^4 t_i^{\text{air}}$
feet air-time variance	$-\text{Var}(t_i^{\text{air}}, t_i^{\text{contact}})$
feet slide	$-\sum_{i=1}^4 \ \mathbf{v}_{\text{foot},i}^{xy}\ c_i$
feet height clearance	$\sum_{i=1}^4 \exp(-(\mathbf{p}_{\text{foot}}^{\text{des},z} + h^{\text{terrain}} - \mathbf{p}_{\text{foot}}^z)^2/0.01))$
feet to hips position	$-\sum_{i=1}^4 \ \tilde{\mathbf{p}}_{\text{foot},i}^{xy} - \tilde{\mathbf{p}}_{\text{hip},i}^{xy}\ $
feet vertical surface contact	$-1(\ \mathbf{f}_{xy}\ > 4 f_z)$

- Make the base align with the terrain (e.g. when on stairs).
- The terrain orientation is taken from the simulator

Each reward are multiplied by a scalar to increase its importance

Type	Expression
base linear velocity	$\exp(-\ \mathbf{v}^{\text{des}} - \mathbf{v}\ ^2/0.05)$
yaw rate	$\exp(-(w_z^{\text{des}} - w_z)^2/0.25)$
base height	$\exp(-(h_{\text{base}}^{\text{des}} + h^{\text{terrain}} - h_{\text{base}})^2/0.01)$
base pitch orientation	$-(\theta^{\text{terrain}} - \theta)^2$
base roll orientation	$-(\phi^{\text{terrain}} - \phi)^2$
base vertical velocity	$-v_z^2$
joints torque	$-\ \boldsymbol{\tau}\ ^2$
joints acceleration	$-\ \ddot{\mathbf{q}}\ ^2$
joints power	$-\ \dot{\mathbf{q}} \odot \boldsymbol{\tau}\ ^2$
action rate	$-\ (\mathbf{a}_k - \mathbf{a}_{k-1})\ ^2$
action smoothness	$-\ (\mathbf{a}_k - 2\mathbf{a}_{k-1} + \mathbf{a}_{k-2})\ ^2$
undesired contact	$-\sum_i 1_{\text{contact},i}$
stance contact suggestion	$\frac{1}{4} \sum_{i=1}^4 c_i$
feet air time	$\sum_{i=1}^4 t_i^{\text{air}}$
feet air-time variance	$-\text{Var}(t_i^{\text{air}}, t_i^{\text{contact}})$
feet slide	$-\sum_{i=1}^4 \ \mathbf{v}_{\text{foot},i}^{xy}\ c_i$
feet height clearance	$\sum_{i=1}^4 \exp(-(\mathbf{p}_{\text{foot}}^{\text{des},z} + h^{\text{terrain}} - \mathbf{p}_{\text{foot}}^z)^2/0.01))$
feet to hips position	$-\sum_{i=1}^4 \ \tilde{\mathbf{p}}_{\text{foot},i}^{xy} - \tilde{\mathbf{p}}_{\text{hip},i}^{xy}\ ^2$
feet vertical surface contact	$-1(\ \mathbf{f}_{xy}\ > 4 f_z)$

Reduce oscillation of the
base in the z direction

Each reward are multiplied by a scalar to increase its importance

Type	Expression
base linear velocity	$\exp(-\ \mathbf{v}^{\text{des}} - \mathbf{v}\ ^2/0.05)$
yaw rate	$\exp(-(w_z^{\text{des}} - w_z)^2/0.25)$
base height	$\exp(-(h_{\text{base}}^{\text{des}} + h^{\text{terrain}} - h_{\text{base}})^2/0.01)$
base pitch orientation	$-(\theta^{\text{terrain}} - \theta)^2$
base roll orientation	$-(\phi^{\text{terrain}} - \phi)^2$
base vertical velocity	$-v_z^2$
joints torque	$-\ \boldsymbol{\tau}\ ^2$
joints acceleration	$-\ \ddot{\mathbf{q}}\ ^2$
joints power	$-\ \dot{\mathbf{q}} \odot \boldsymbol{\tau}\ ^2$
action rate	$-\ (\mathbf{a}_k - \mathbf{a}_{k-1})\ ^2$
action smoothness	$-\ (\mathbf{a}_k - 2\mathbf{a}_{k-1} + \mathbf{a}_{k-2})\ ^2$
undesired contact	$-\sum_i 1_{\text{contact},i}$
stance contact suggestion	$\frac{1}{4} \sum_{i=1}^4 c_i$
feet air time	$\sum_{i=1}^4 t_i^{\text{air}}$
feet air-time variance	$-\text{Var}(t_i^{\text{air}}, t_i^{\text{contact}})$
feet slide	$-\sum_{i=1}^4 \ \mathbf{v}_{\text{foot},i}^{xy}\ c_i$
feet height clearance	$\sum_{i=1}^4 \exp(-(\mathbf{p}_{\text{foot}}^{\text{des},z} + h^{\text{terrain}} - \mathbf{p}_{\text{foot}}^z)^2/0.01))$
feet to hips position	$-\sum_{i=1}^4 \ \tilde{\mathbf{p}}_{\text{foot},i}^{xy} - \tilde{\mathbf{p}}_{\text{hip},i}^{xy}\ $
feet vertical surface contact	$-1(\ \mathbf{f}_{xy}\ > 4 f_z)$

- Smoother motions
- Less power/torque used

Each reward are multiplied by a scalar to increase its importance

Type	Expression
base linear velocity	$\exp(-\ \mathbf{v}^{\text{des}} - \mathbf{v}\ ^2/0.05)$
yaw rate	$\exp(-(w_z^{\text{des}} - w_z)^2/0.25)$
base height	$\exp(-(h_{\text{base}}^{\text{des}} + h^{\text{terrain}} - h_{\text{base}})^2/0.01)$
base pitch orientation	$-(\theta^{\text{terrain}} - \theta)^2$
base roll orientation	$-(\phi^{\text{terrain}} - \phi)^2$
base vertical velocity	$-v_z^2$
joints torque	$-\ \boldsymbol{\tau}\ ^2$
joints acceleration	$-\ \ddot{\mathbf{q}}\ ^2$
joints power	$- \dot{\mathbf{q}} \odot \boldsymbol{\tau} $
action rate	$-\ (\mathbf{a}_k - \mathbf{a}_{k-1})\ ^2$
action smoothness	$-\ (\mathbf{a}_k - 2\mathbf{a}_{k-1} + \mathbf{a}_{k-2})\ ^2$
undesired contact	$-\sum_i 1_{\text{contact},i}$
stance contact suggestion	$\frac{1}{4} \sum_{i=1}^4 c_i$
feet air time	$\sum_{i=1}^4 t_i^{\text{air}}$
feet air-time variance	$-\text{Var}(t_i^{\text{air}}, t_i^{\text{contact}})$
feet slide	$-\sum_{i=1}^4 \ \mathbf{v}_{\text{foot},i}^{xy}\ c_i$
feet height clearance	$\sum_{i=1}^4 \exp(-(\mathbf{p}_{\text{foot}}^{\text{des},z} + h^{\text{terrain}} - \mathbf{p}_{\text{foot}}^z)^2/0.01))$
feet to hips position	$-\sum_{i=1}^4 \ \tilde{\mathbf{p}}_{\text{foot},i}^{xy} - \tilde{\mathbf{p}}_{\text{hip},i}^{xy}\ $
feet vertical surface contact	$-1(\ \mathbf{f}_{xy}\ > 4 f_z)$

Real robot actuator cannot
change setpoint instantaneously!

Each reward are multiplied by a scalar to increase its importance

Type	Expression
base linear velocity	$\exp(-\ \mathbf{v}^{\text{des}} - \mathbf{v}\ ^2/0.05)$
yaw rate	$\exp(-(w_z^{\text{des}} - w_z)^2/0.25)$
base height	$\exp(-(h_{\text{base}}^{\text{des}} + h^{\text{terrain}} - h_{\text{base}})^2/0.01)$
base pitch orientation	$-(\theta^{\text{terrain}} - \theta)^2$
base roll orientation	$-(\phi^{\text{terrain}} - \phi)^2$
base vertical velocity	$-v_z^2$
joints torque	$-\ \boldsymbol{\tau}\ ^2$
joints acceleration	$-\ \ddot{\mathbf{q}}\ ^2$
joints power	$- \dot{\mathbf{q}} \odot \boldsymbol{\tau} $
action rate	$-\ (\mathbf{a}_k - \mathbf{a}_{k-1})\ ^2$
action smoothness	$-\ (\mathbf{a}_k - 2\mathbf{a}_{k-1} + \mathbf{a}_{k-2})\ ^2$
undesired contact	$-\sum_i 1_{\text{contact},i}$
stance contact suggestion	$\frac{1}{4} \sum_{i=1}^4 c_i$
feet air time	$\sum_{i=1}^4 t_i^{\text{air}}$
feet air-time variance	$-\text{Var}(t_i^{\text{air}}, t_i^{\text{contact}})$
feet slide	$-\sum_{i=1}^4 \ \mathbf{v}_{\text{foot},i}^{xy}\ c_i$
feet height clearance	$\sum_{i=1}^4 \exp(-(\mathbf{p}_{\text{foot}}^{\text{des},z} + h^{\text{terrain}} - \mathbf{p}_{\text{foot}}^z)^2/0.01))$
feet to hips position	$-\sum_{i=1}^4 \ \tilde{\mathbf{p}}_{\text{foot},i}^{xy} - \tilde{\mathbf{p}}_{\text{hip},i}^{xy}\ $
feet vertical surface contact	$-1(\ \mathbf{f}_{xy}\ > 4 f_z)$

Try not to hit with the trunk/part of the legs the environment

Each reward are multiplied by a scalar to increase its importance

Type	Expression
base linear velocity	$\exp(-\ \mathbf{v}^{\text{des}} - \mathbf{v}\ ^2/0.05)$
yaw rate	$\exp(-(w_z^{\text{des}} - w_z)^2/0.25)$
base height	$\exp(-(h_{\text{base}}^{\text{des}} + h^{\text{terrain}} - h_{\text{base}})^2/0.01)$
base pitch orientation	$-(\theta^{\text{terrain}} - \theta)^2$
base roll orientation	$-(\phi^{\text{terrain}} - \phi)^2$
base vertical velocity	$-v_z^2$
joints torque	$-\ \boldsymbol{\tau}\ ^2$
joints acceleration	$-\ \ddot{\mathbf{q}}\ ^2$
joints power	$- \dot{\mathbf{q}} \odot \boldsymbol{\tau} $
action rate	$-\ (\mathbf{a}_k - \mathbf{a}_{k-1})\ ^2$
action smoothness	$-\ (\mathbf{a}_k - 2\mathbf{a}_{k-1} + \mathbf{a}_{k-2})\ ^2$
undesired contact	$-\sum_i 1_{\text{contact},i}$
stance contact suggestion	$\frac{1}{4} \sum_{i=1}^4 c_i$
feet air time	$\sum_{i=1}^4 t_i^{\text{air}}$
feet air-time variance	$-\text{Var}(t_i^{\text{air}}, t_i^{\text{contact}})$
feet slide	$-\sum_{i=1}^4 \ \mathbf{v}_{\text{foot},i}^{xy}\ c_i$
feet height clearance	$\sum_{i=1}^4 \exp(-(\mathbf{p}_{\text{foot}}^{\text{des},z} + h^{\text{terrain}} - \mathbf{p}_{\text{foot}}^z)^2/0.01))$
feet to hips position	$-\sum_{i=1}^4 \ \tilde{\mathbf{p}}_{\text{foot},i}^{xy} - \tilde{\mathbf{p}}_{\text{hip},i}^{xy}\ $
feet vertical surface contact	$-1(\ \mathbf{f}_{xy}\ > 4 f_z)$

- When stopped, try to maintain all the foot on the ground
- Try to not step too fast
- Try to maintain symmetry in stepping (left foot steps as fast as the right one)

Each reward are multiplied by a scalar to increase its importance

Type	Expression
base linear velocity	$\exp(-\ \mathbf{v}^{\text{des}} - \mathbf{v}\ ^2/0.05)$
yaw rate	$\exp(-(w_z^{\text{des}} - w_z)^2/0.25)$
base height	$\exp(-(h_{\text{base}}^{\text{des}} + h^{\text{terrain}} - h_{\text{base}})^2/0.01)$
base pitch orientation	$-(\theta^{\text{terrain}} - \theta)^2$
base roll orientation	$-(\phi^{\text{terrain}} - \phi)^2$
base vertical velocity	$-v_z^2$
joints torque	$-\ \boldsymbol{\tau}\ ^2$
joints acceleration	$-\ \ddot{\mathbf{q}}\ ^2$
joints power	$- \dot{\mathbf{q}} \odot \boldsymbol{\tau} $
action rate	$-\ (\mathbf{a}_k - \mathbf{a}_{k-1})\ ^2$
action smoothness	$-\ (\mathbf{a}_k - 2\mathbf{a}_{k-1} + \mathbf{a}_{k-2})\ ^2$
undesired contact	$-\sum_i 1_{\text{contact},i}$
stance contact suggestion	$\frac{1}{4} \sum_{i=1}^4 c_i$
feet air time	$\sum_{i=1}^4 t_i^{\text{air}}$
feet air-time variance	$-\text{Var}(t_i^{\text{air}}, t_i^{\text{contact}})$
feet slide	$-\sum_{i=1}^4 \ \mathbf{v}_{\text{foot},i}^{xy}\ c_i$
feet height clearance	$\sum_{i=1}^4 \exp(-(\mathbf{p}_{\text{foot},i}^{\text{des},z} + h^{\text{terrain}} - \mathbf{p}_{\text{foot},i}^z)^2/0.01))$
feet to hips position	$-\sum_{i=1}^4 \ \tilde{\mathbf{p}}_{\text{foot},i}^{xy} - \tilde{\mathbf{p}}_{\text{hip},i}^{xy}\ $
feet vertical surface contact	$-1(\ \mathbf{f}_{xy}\ > 4 f_z)$

- Avoid sliding the foot (too much sim2real)
- Reach the step height

Each reward are multiplied by a scalar to increase its importance

Type	Expression
base linear velocity	$\exp(-\ \mathbf{v}^{\text{des}} - \mathbf{v}\ ^2/0.05)$
yaw rate	$\exp(-(w_z^{\text{des}} - w_z)^2/0.25)$
base height	$\exp(-(h_{\text{base}}^{\text{des}} + h^{\text{terrain}} - h_{\text{base}})^2/0.01)$
base pitch orientation	$-(\theta^{\text{terrain}} - \theta)^2$
base roll orientation	$-(\phi^{\text{terrain}} - \phi)^2$
base vertical velocity	$-v_z^2$
joints torque	$-\ \boldsymbol{\tau}\ ^2$
joints acceleration	$-\ \ddot{\mathbf{q}}\ ^2$
joints power	$- \dot{\mathbf{q}} \odot \boldsymbol{\tau} $
action rate	$-\ (\mathbf{a}_k - \mathbf{a}_{k-1})\ ^2$
action smoothness	$-\ (\mathbf{a}_k - 2\mathbf{a}_{k-1} + \mathbf{a}_{k-2})\ ^2$
undesired contact	$-\sum_i 1_{\text{contact},i}$
stance contact suggestion	$\frac{1}{4} \sum_{i=1}^4 c_i$
feet air time	$\sum_{i=1}^4 t_i^{\text{air}}$
feet air-time variance	$-\text{Var}(t_i^{\text{air}}, t_i^{\text{contact}})$
feet slide	$-\sum_{i=1}^4 \ \mathbf{v}_{\text{foot},i}^{xy}\ c_i$
feet height clearance	$\sum_{i=1}^4 \exp(-(\mathbf{p}_{\text{foot},i}^{\text{des},z} + h^{\text{terrain}} - \mathbf{p}_{\text{foot},i}^z)^2/0.01)$
feet to hips position	$-\sum_{i=1}^4 \ \tilde{\mathbf{p}}_{\text{foot},i}^{xy} - \tilde{\mathbf{p}}_{\text{hip},i}^{xy}\ $
feet vertical surface contact	$-1(\ \mathbf{f}_{xy}\ > 4 f_z)$

Try to maintain the foot position near the hips

Each reward are multiplied by a scalar to increase its importance

Type	Expression
base linear velocity	$\exp(-\ \mathbf{v}^{\text{des}} - \mathbf{v}\ ^2/0.05)$
yaw rate	$\exp(-(w_z^{\text{des}} - w_z)^2/0.25)$
base height	$\exp(-(h_{\text{base}}^{\text{des}} + h^{\text{terrain}} - h_{\text{base}})^2/0.01)$
base pitch orientation	$-(\theta^{\text{terrain}} - \theta)^2$
base roll orientation	$-(\phi^{\text{terrain}} - \phi)^2$
base vertical velocity	$-v_z^2$
joints torque	$-\ \boldsymbol{\tau}\ ^2$
joints acceleration	$-\ \ddot{\mathbf{q}}\ ^2$
joints power	$- \dot{\mathbf{q}} \odot \boldsymbol{\tau} $
action rate	$-\ (\mathbf{a}_k - \mathbf{a}_{k-1})\ ^2$
action smoothness	$-\ (\mathbf{a}_k - 2\mathbf{a}_{k-1} + \mathbf{a}_{k-2})\ ^2$
undesired contact	$-\sum_i 1_{\text{contact},i}$
stance contact suggestion	$\frac{1}{4} \sum_{i=1}^4 c_i$
feet air time	$\sum_{i=1}^4 t_i^{\text{air}}$
feet air-time variance	$-\text{Var}(t_i^{\text{air}}, t_i^{\text{contact}})$
feet slide	$-\sum_{i=1}^4 \ \mathbf{v}_{\text{foot},i}^{xy}\ c_i$
feet height clearance	$\sum_{i=1}^4 \exp(-(\mathbf{p}_{\text{foot}}^{\text{des},z} + h^{\text{terrain}} - \mathbf{p}_{\text{foot}}^z)^2/0.01))$
feet to hips position	$-\sum_{i=1}^4 \ \tilde{\mathbf{p}}_{\text{foot},i}^{xy} - \tilde{\mathbf{p}}_{\text{hip},i}^{xy}\ $
feet vertical surface contact	$-1(\ \mathbf{f}_{xy}\ > 4 f_z)$

When hitting the obstacle, try to not put too much force

Each reward are multiplied by a scalar to increase its importance

Type	Expression
base linear velocity	$\exp(-\ \mathbf{v}^{\text{des}} - \mathbf{v}\ ^2/0.05)$
yaw rate	$\exp(-(w_z^{\text{des}} - w_z)^2/0.25)$
base height	$\exp(-(h_{\text{base}}^{\text{des}} + h^{\text{terrain}} - h_{\text{base}})^2/0.01)$
base pitch orientation	$-(\theta^{\text{terrain}} - \theta)^2$
base roll orientation	$-(\phi^{\text{terrain}} - \phi)^2$
base vertical velocity	$-v_z^2$
joints torque	$-\ \boldsymbol{\tau}\ ^2$
joints acceleration	$-\ \ddot{\mathbf{q}}\ ^2$
joints power	$- \dot{\mathbf{q}} \odot \boldsymbol{\tau} $
action rate	$-\ (\mathbf{a}_k - \mathbf{a}_{k-1})\ ^2$
action smoothness	$-\ (\mathbf{a}_k - 2\mathbf{a}_{k-1} + \mathbf{a}_{k-2})\ ^2$
undesired contact	$-\sum_i 1_{\text{contact},i}$
stance contact suggestion	$\frac{1}{4} \sum_{i=1}^4 c_i$
feet air time	$\sum_{i=1}^4 t_i^{\text{air}}$
feet air-time variance	$-\text{Var}(t_i^{\text{air}}, t_i^{\text{contact}})$
feet slide	$-\sum_{i=1}^4 \ \mathbf{v}_{\text{foot},i}^{xy}\ c_i$
feet height clearance	$\sum_{i=1}^4 \exp(-(\mathbf{p}_{\text{foot}}^{\text{des},z} + h^{\text{terrain}} - \mathbf{p}_{\text{foot}}^z)^2/0.01))$
feet to hips position	$-\sum_{i=1}^4 \ \tilde{\mathbf{p}}_{\text{foot},i}^{xy} - \tilde{\mathbf{p}}_{\text{hip},i}^{xy}\ $
feet vertical surface contact	$-1(\mathbf{f}_{xy} > 4 f_z)$

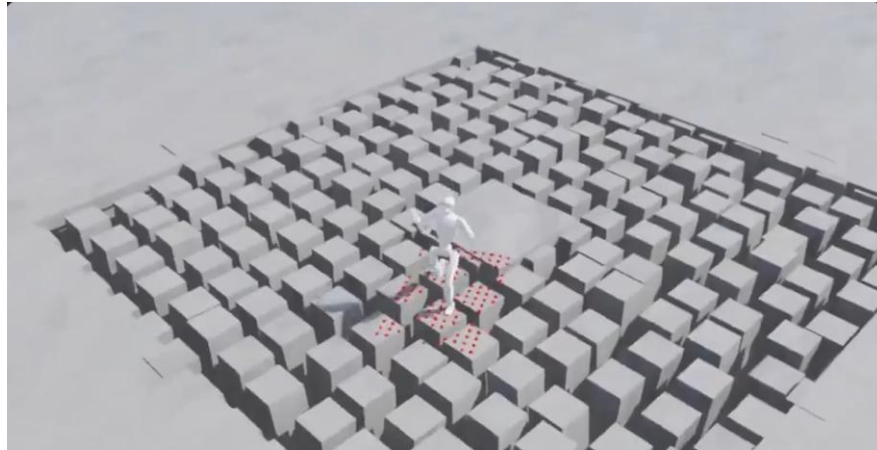
- Rewards can be even discontinuous (if and else)
- Tuning can be tedious, **because rewards usually fight each other**
- GPU simulators helps to iterate fast

- Example of quadruped rewards in real-life



<https://github.com/iit-DLSLab/basic-locomotion-isaacsim>

- Quadruped rewards are in general easier to tune (12 joints)
- Humanoids are very difficult. **Which is the right posture to have?**



video taken from X

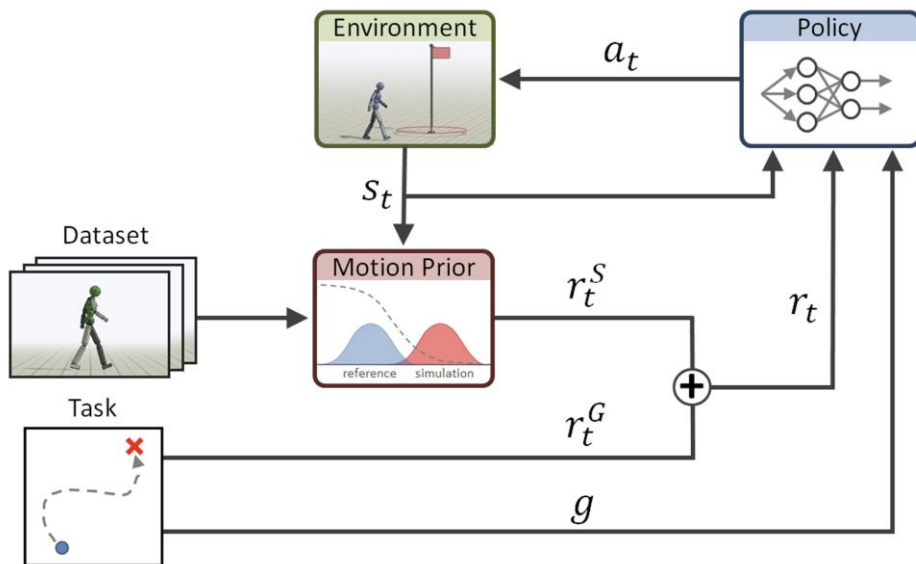


Humanoid olympics

Type	Expression
base linear velocity	$\exp(-\ \mathbf{v}^{\text{des}} - \mathbf{v}\ ^2/0.05)$
yaw rate	$\exp(-(w_z^{\text{des}} - w_z)^2/0.25)$
base height	$\exp(-(h_{\text{base}}^{\text{des}} + h_{\text{terrain}} - h_{\text{base}})^2/0.01)$
base pitch orientation	$(\theta_{\text{terrain}} - \theta)^2$
base roll orientation	$(\phi_{\text{terrain}} - \phi)^2$
base vertical velocity	$-v_z^2$
joints torque	$\ \boldsymbol{\tau} \odot \mathbf{f}\ ^2$
joints acceleration	$\ \ddot{\mathbf{q}} \odot \mathbf{f}\ ^2$
joints power	$\dot{\mathbf{q}} \odot \boldsymbol{\tau} \odot \mathbf{f}$
action rate	$\ (\mathbf{a}_k - \mathbf{a}_{k-1}) \odot \mathbf{f}\ ^2$
action smoothness	$\ (\mathbf{a}_k - 2\mathbf{a}_{k-1} + \mathbf{a}_{k-2}) \odot \mathbf{f}\ ^2$
undesired contact	$\sum_i 1_{\text{contact},i}$
stance contact suggestion	$\frac{1}{4} \sum_{i=1}^4 c_i$
feet air time	$\sum_{i=1}^4 t_{\text{air},i}$
feet air time variance	$\text{Var}(t_{\text{air}}, t_{\text{contact}})$
feet slide	$\sum_{i=1}^4 \ \mathbf{v}_{\text{foot},i}^{xy}\ c_i$
feet height clearance	$\sum_{i=1}^4 \exp(-(\mathbf{p}_{\text{foot}}^{\text{des},z} + h_{\text{terrain}} - \mathbf{p}_{\text{foot}}^z)^2/0.01))$
feet to hips position	$\sum_{i=1}^4 \ \tilde{\mathbf{p}}_{\text{foot},i}^{xy} - \tilde{\mathbf{p}}_{\text{hip},i}^{xy}\$
feet vertical surface contact	$1(\ \mathbf{f}_{xy}\ > 4 f_z)$

- Only task rewards are considered
- Collect data from motion capture data/animation
- Train a network that compare the policy trajectory and the dataset

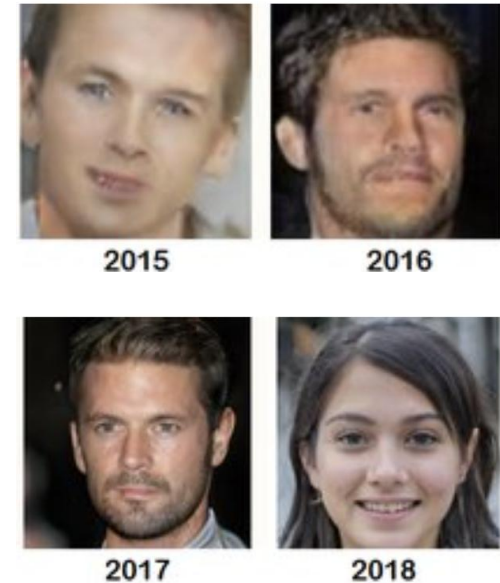
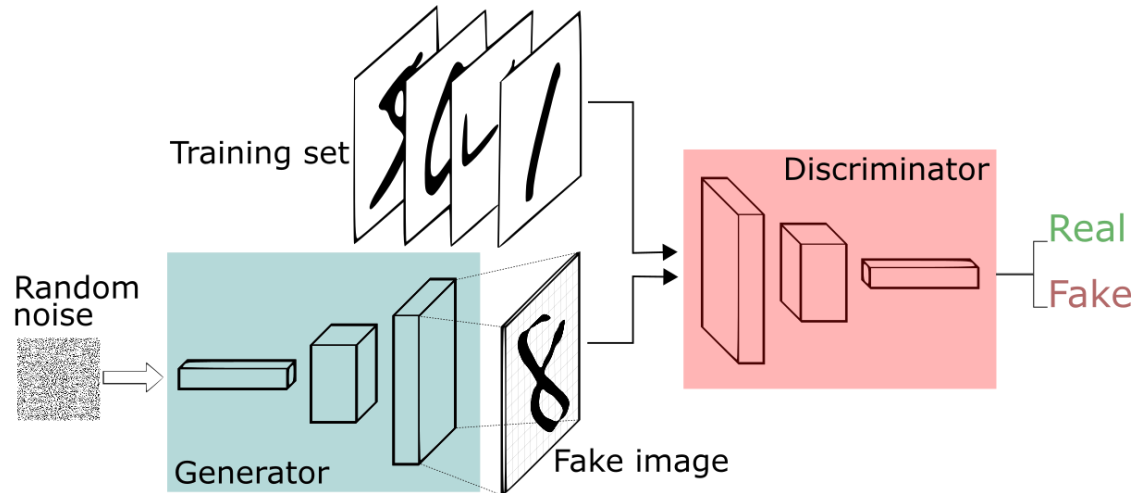
$$r = r_{\text{task}} + r_{\text{style}}$$



- Only task rewards are considered
- Collect data from motion capture data/animation
- Train a network that compare the policy trajectory and the dataset

$$r = r_{task} + r_{style}$$

Goal: generate data that is indistinguishable wrt a real one



GAN Network



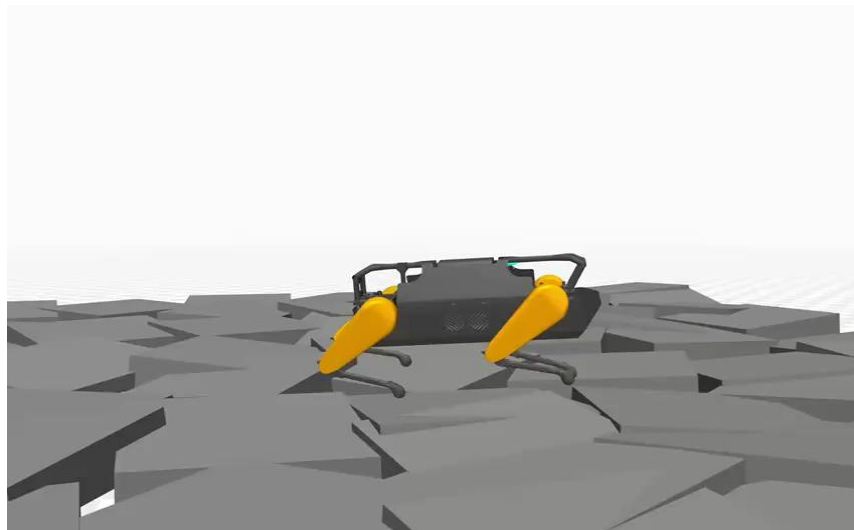
Generator is the policy

- Only task rewards are considered
- Collect data from motion capture data/animation
- Train a network that compare the policy trajectory and the dataset

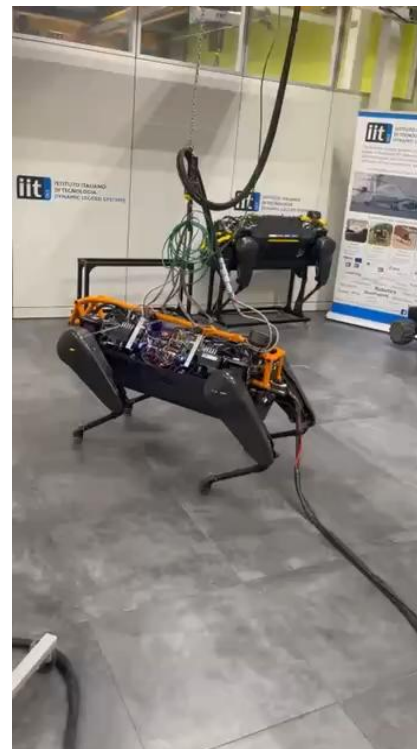
$$r = r_{task} + r_{style}$$

- Environments
- Observations
- Reward Functions
- **From Simulators to Real World**
 - Offline Identification
 - Online Identification

Simulation

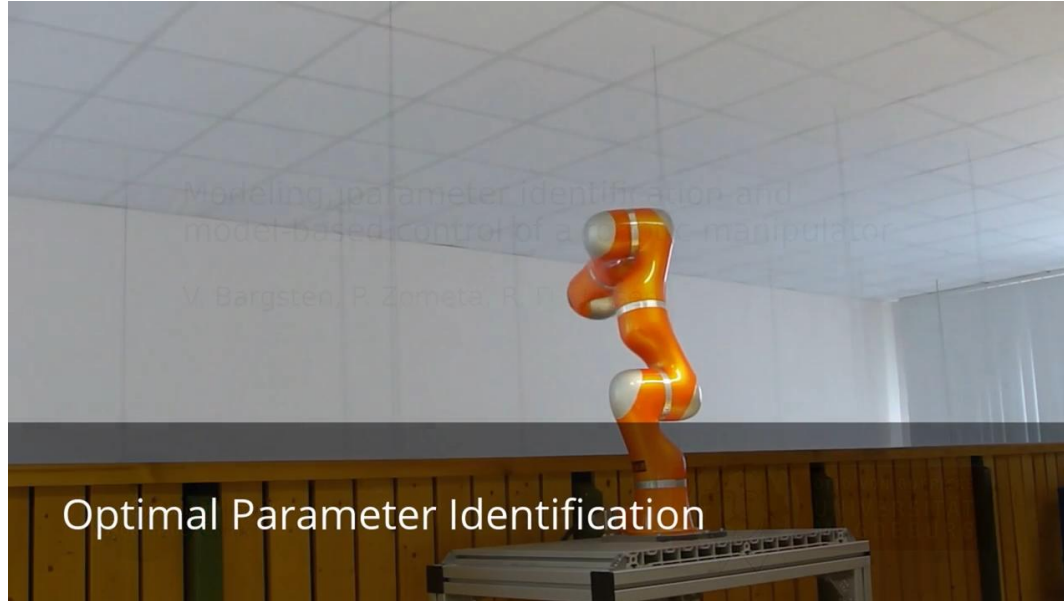


Real world



What is the source of the problem?

Simulation can be different from reality!



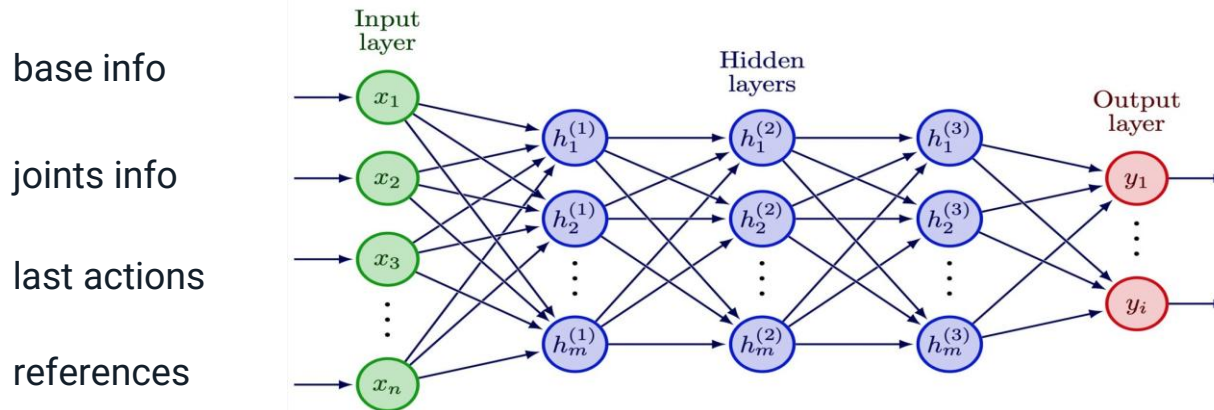
1. Collect trajectories on the real robot

2. Perform a least squares



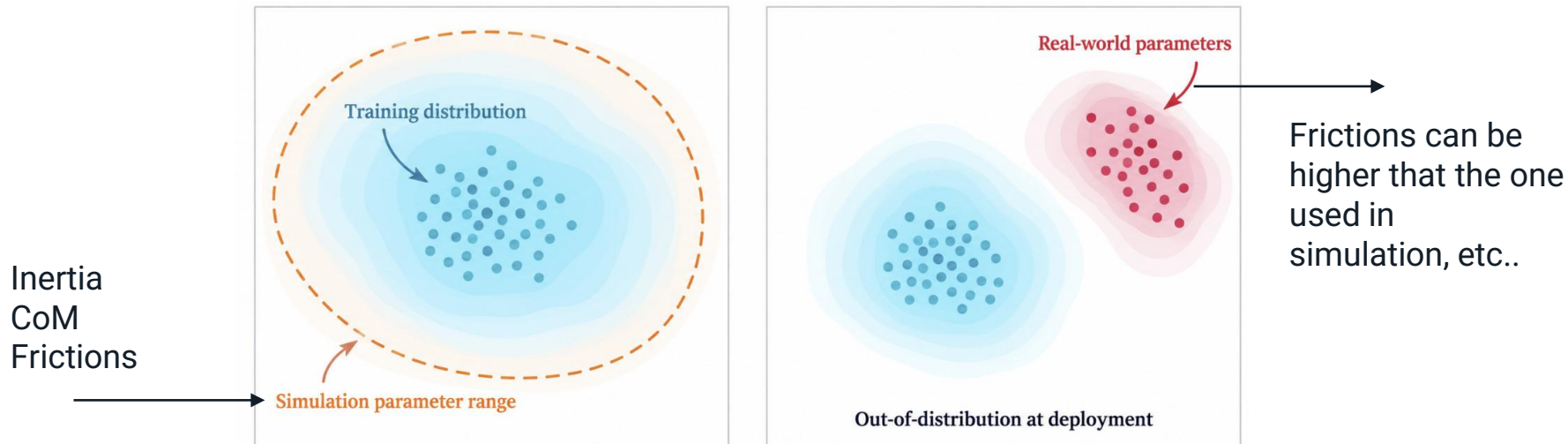
New dynamical parameters to do control

- What's the source of the problem? **Simulation != Reality**
 - Model-based methods gives "**flexibility**" during deployment
 - RL methods instead "**does not**" – the problem should be solved in the simulation



Actions = Desired joints position to feed a **PD controller**

- The problem is data distribution!



The simulation should be made similar to reality, not viceversa (e.g. better motor tuning helps up to a point)!

Source of problems:

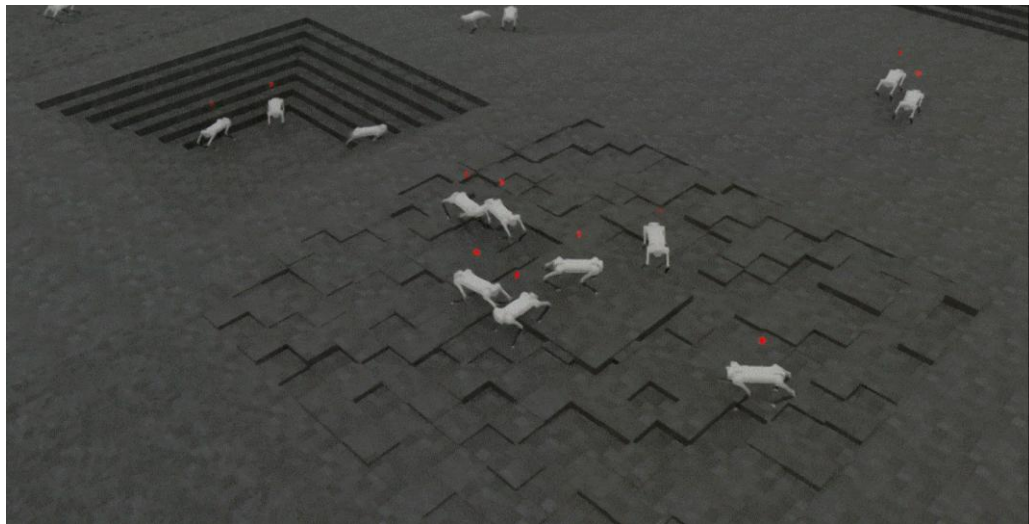
- Bias and noise in parameters estimation
- Time changing parameters (e.g. wear of gearbox, added payload)
- Unknown nonlinearities
- Imperfect contact dynamics (soft contact, hard contact, ..)
- Discretization error of simulators
-

Different solutions exists:

- **Preliminary interaction with the hardware**
 - Sampling-based identification
- **Simulation only**
 - Domain Randomization
 - Rapid Motor Adaptation

Different solutions exist:

- **Preliminary interaction with the hardware**
 - Sampling-based identification
- **Simulation only**
 - **Domain Randomization**
 - Rapid Motor Adaptation



During training **each robot is unique**, e.g. has a different parameters

- Frictions
- Gains
- Input noise..

- **The policy learns to be robust**, but can be not optimal for a specific set of parameters!
- if you randomize too much (e.g. Mass from 5kg to 80), the **performance will be bad**.

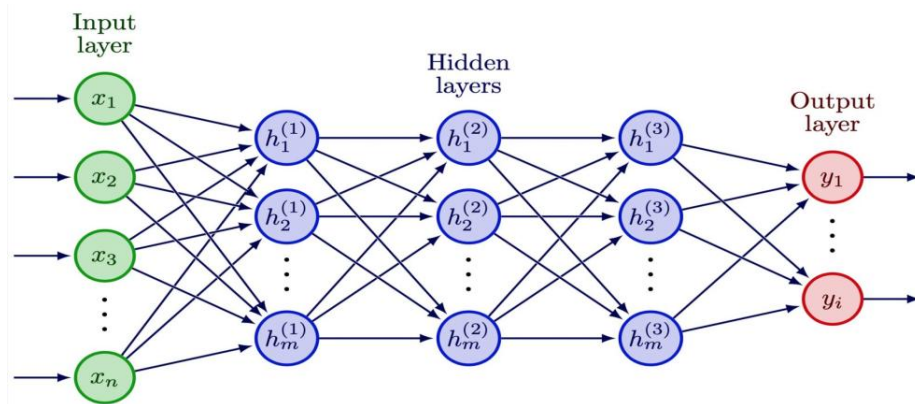
How to increase optimality?

base info

joints info

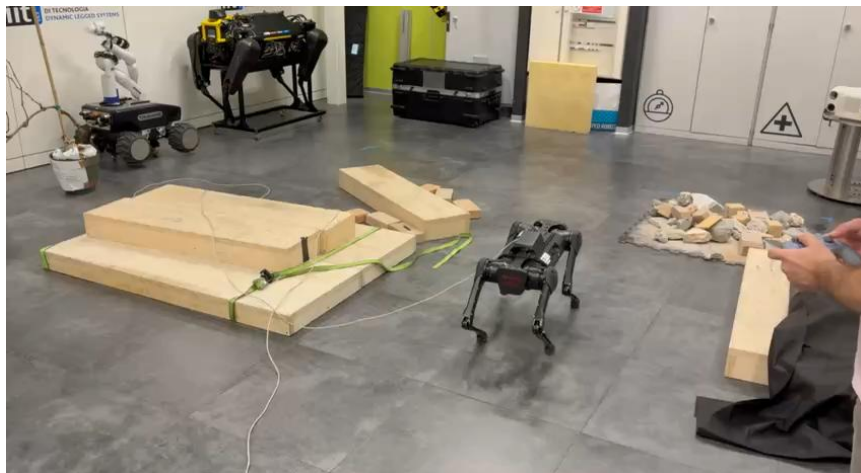
last actions

references

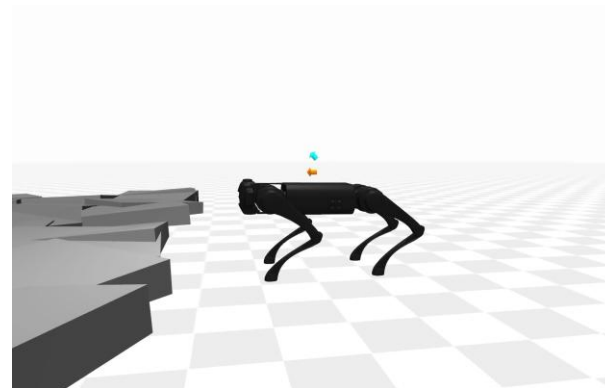
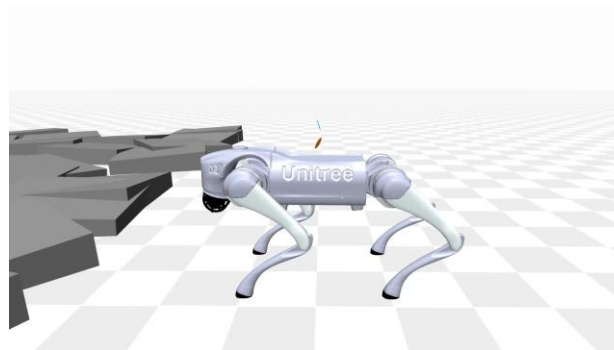


Actions = Desired joints
position to feed a **PD**
controller

Domain Randomization



Same policy, different robots



PRO:

- Easy to implement
- Policy is robust to deployment (up to a certain point)

CONS:

- A robust policy is not optimal
- More you randomize (because you don't know the true value), worse are the performance

Different solutions exists:

- **Preliminary interaction with the hardware**
 - **Sampling-based identification**
- **Simulation only**
 - **Domain Randomization**
 - **Rapid Motor Adaptation**

Idea: Let's try to identified the simulation parameters, in order to make it evolve like the real robot! **This is before the learning start.**



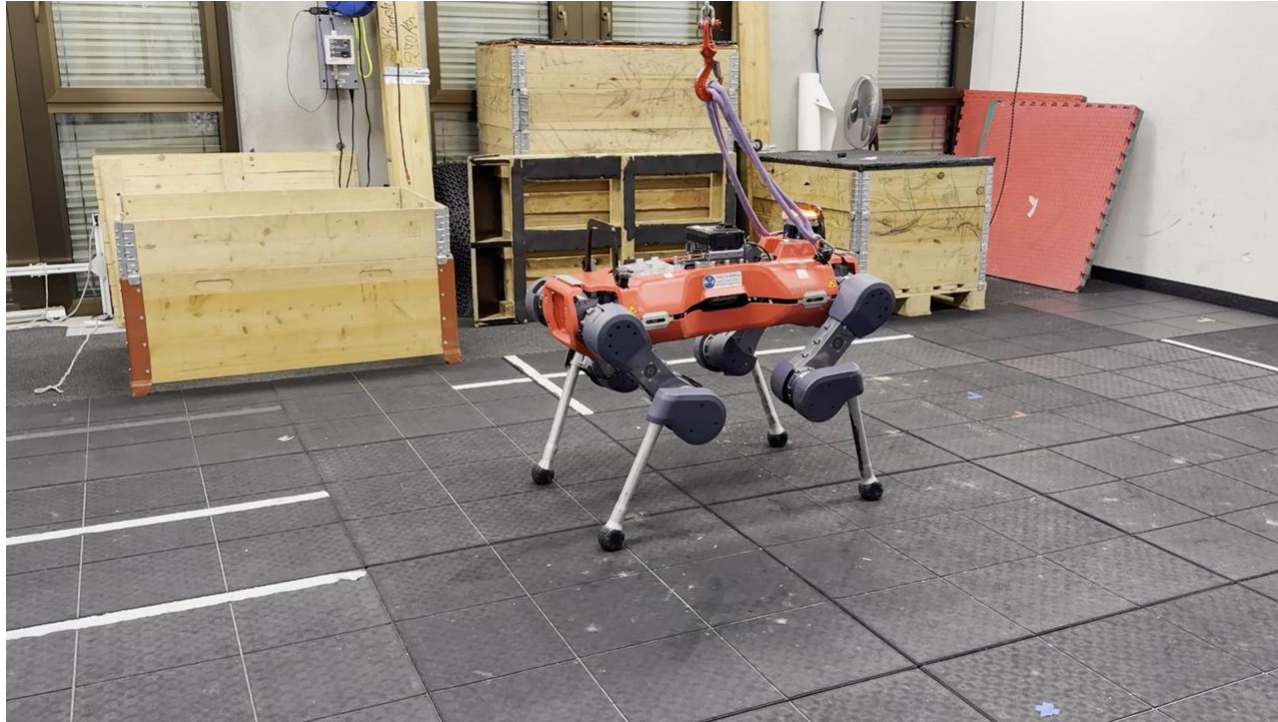
Similar to model-based methods, but with the goal to optimize the simulation:

- **Collect trajectories on the real robot, and make simulations match it!**

For each simulated robot:

- Sample a set of parameters
- Compute **joint errors** wrt real world
- Take the best candidate





For each simulated robot:

- Sample a set of parameters
- Compute the error wrt real world
- Take the best candidate (>1)



This is the new (smaller!) distribution bound for domain randomization

PRO:

- Easy to implement
- Simulation is very closed to reality -> policy is more optimal

CONS:

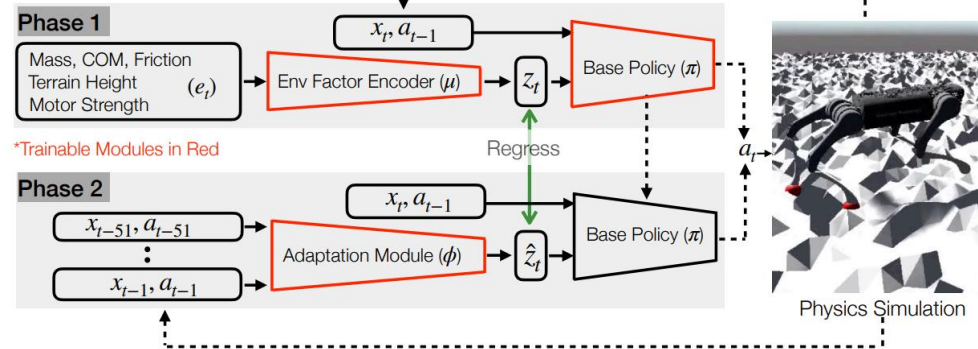
- Works with the robot joints sensor
- Complicate to estimate parameters that requires movements

Different solutions exist:

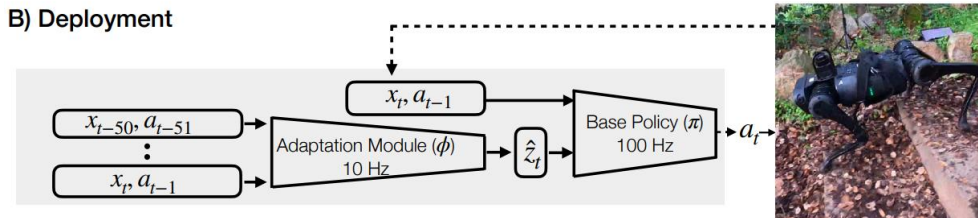
- **Preliminary interaction with the hardware**
 - **Sampling-based identification**
- **Simulation only**
 - **Domain Randomization**
 - **Rapid Motor Adaptation**

RMA learns everything in simulation – **no need of interaction with the hardware first!**

A) Training in Simulation

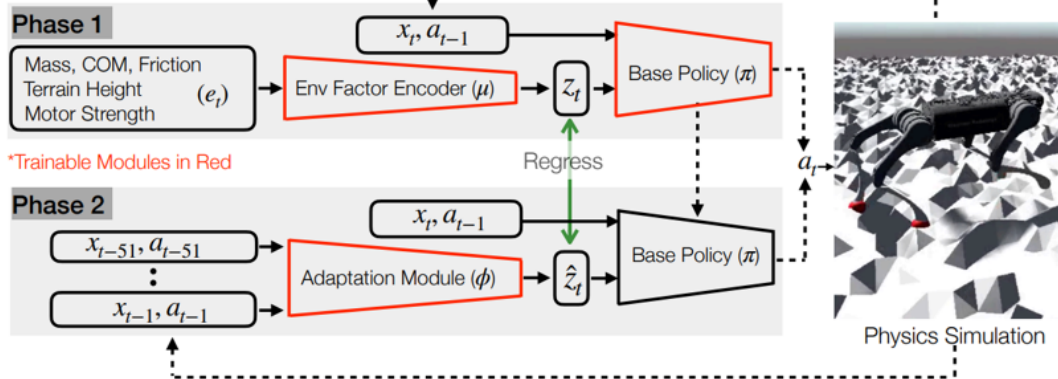


B) Deployment



RMA learns everything in simulation – **no need of interaction with the hardware first!**

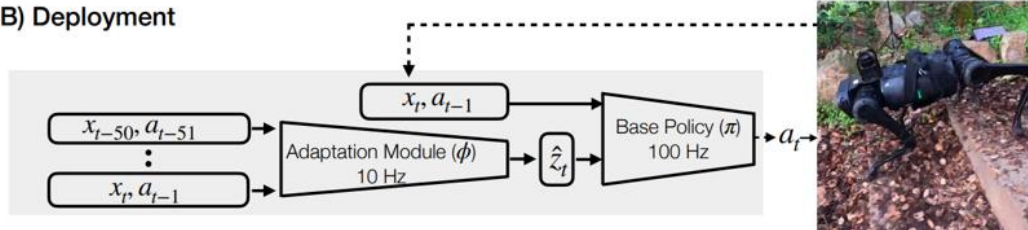
A) Training in Simulation



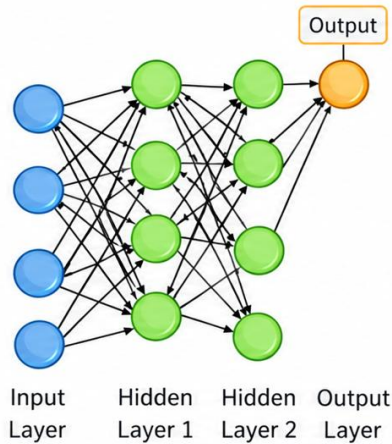
- **During training**, a supervised learning network can be trained to predict ground-truth information
- This prediction **enters in the policy observation!**

RMA learns everything in simulation – **no need of interaction with the hardware first!**

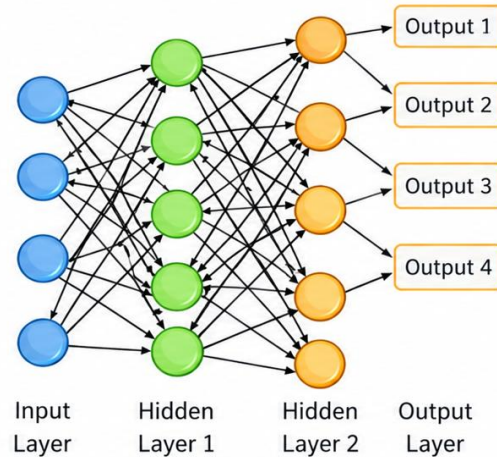
B) Deployment



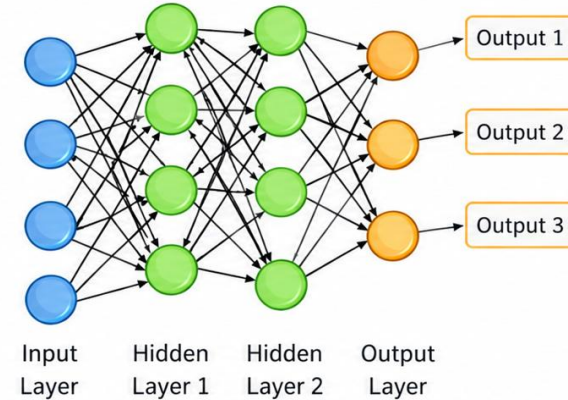
- **During deployment,** this networks is just queried



Critic:
Only used in simulation!



Actor:
**Used in simulation
and in deploy**



RMA:
**Used in simulation and in
deploy.** Predict ground truth info

Initialize policy π , critic V , and state estimator SE

for each PPO training iteration:

collect rollouts in simulation:

observe proprioception: o_t

predict linear velocity: $\hat{z}_t = SE(o_t)$

build policy input: $x_t = [o_t, z_t]$

sample action: $a_t = \pi(x_t)$

step simulation and store transition

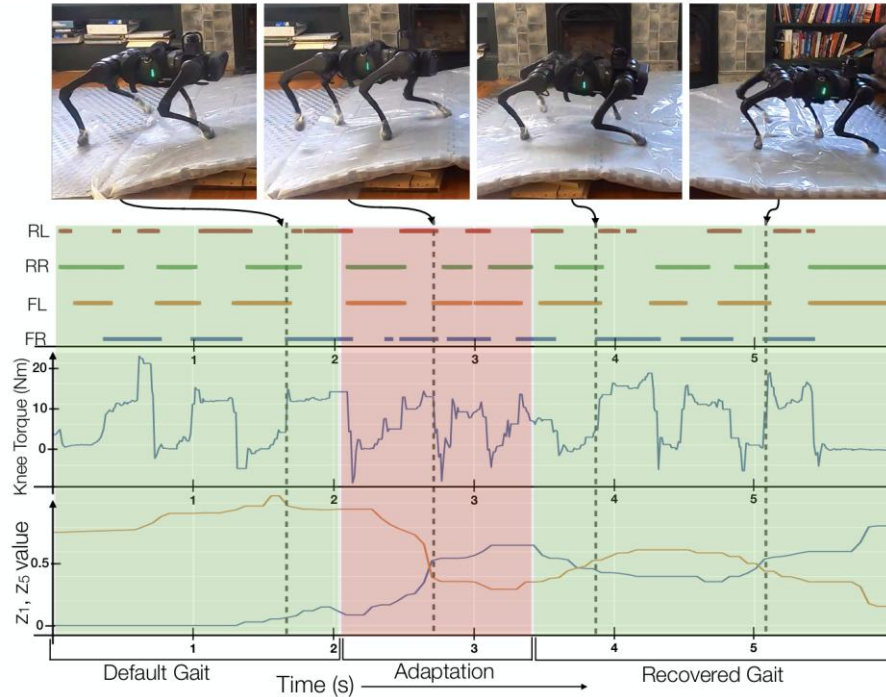
store ground truth from simulation z_t

compute rewards, returns, and advantages

update PPO using collected rollouts

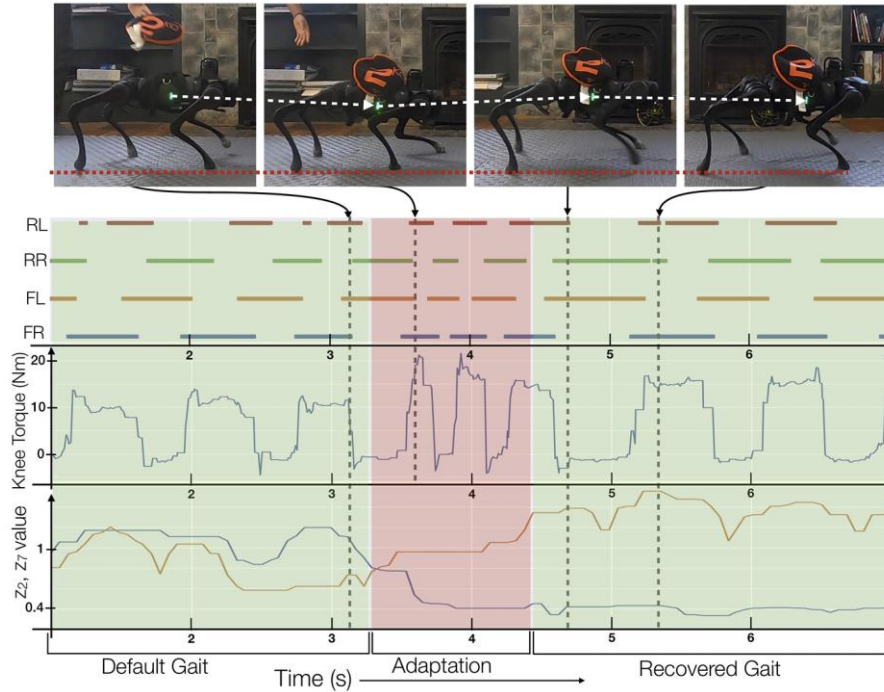
update estimator with supervised loss: $||\hat{z}_t - z_t||$

RMA learns everything in simulation – **no need of interaction with the hardware first!**



Foot slippage experiment

RMA learns everything in simulation – **no need of interaction with the hardware first!**



External payload experiment

PRO:

- to be done only in simulation – no need of hardware interaction
- it can estimate exteroceptive signals

CONS:

- it can get sometimes confused...

1. Most of the methods are only for proprioceptive identification
2. Some methods require preliminary interaction with hardware

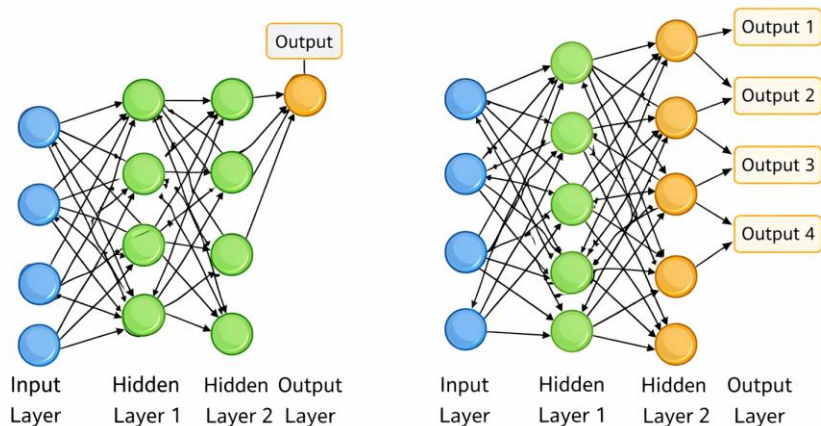
Typical easiest/best procedure:

- Combine Sampling-based sys-id + Domain Randomization

BONUS SLIDES

Easy tricks to increase performances

Remember standard PPO



Critic: $V(s)$

Actor: $\pi(s)$

Only used in simulation! Used in simulation and deploy

They all have the same inputs!

Why don't we use additional info to
train a **better critic**?

Example of better info:

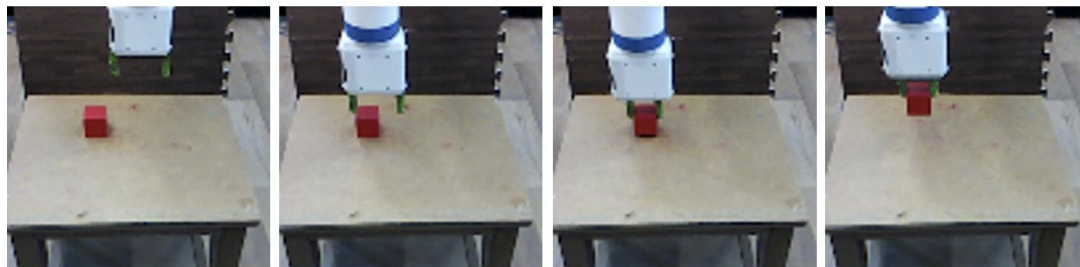
- Actor is trained with noise in the obs to be robust in the real world

$$a_t = \pi(o_t), o_t = s_t + \epsilon$$

- Critic can be training with **noise-free** obs

$$V(s_t)$$

Example of better info:



- Actor is trained with camera images

$$a_t = \pi(I_t), I_t \in R^{50 \times 50}$$

- Critic can be trained with **explicit** object position (privileged information)

$$V(x_t), x_t \in R^3$$

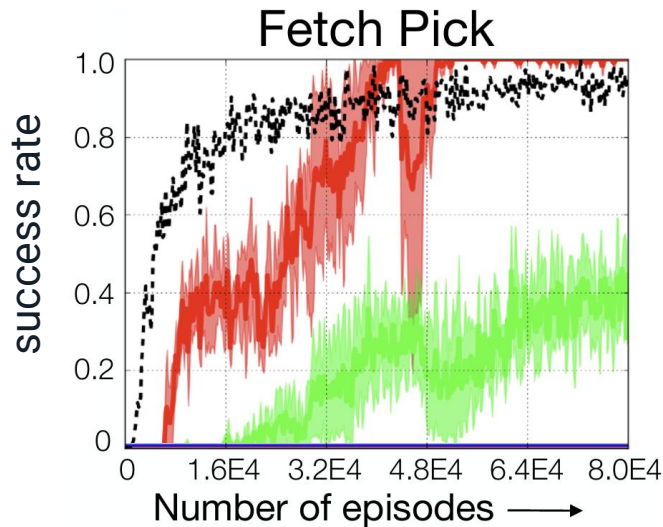
Example of better info:

- Actor is trained with camera images

$$a_t = \pi(I_t), I_t \in R^{50 \times 50}$$

- Critic can be trained with **explicit** object position (privileged information)

$$V(x_t), x_t \in R^3$$



Example of better info for **locomotion**:

- Actor is trained with (**see previous slide**)

$$a_t = \pi(o_t), o_t = s_t + \epsilon$$

- Critic can be trained with **explicit** robot friction, mass, disturbances..

$$V(s_t, e_t), e_t = [\text{friction, mass, ...}]$$

- RL methods are mature for real-world application in locomotion
- ...but not so much for manipulation (**too much exploration to do**)
- learning algorithms are frozen to 2017 (PPO), still a lot of benefits came in these years from tricks (learned state estimator, asymmetric PPO) or better compute/simulators
- .. the next algorithm will probably make a huge jump in the field

Feedback pls!



Any question, or feedback on the the slide/lesson/school is welcome!
Completely anonymous!